

Mizar Evo

Readable, AI-Ready, Scalable Formal Mathematics

Eight problems, eight proposals

A discussion with the Bialystok Mizar team, September 2026

presenter notes edition

Mizar Evo project

September 2026

Read aloud:

- **Thank you for inviting me to Bialystok. Today I will introduce Mizar Evo.**
- **Mizar is a system for writing mathematical proofs and checking them by computer. Evo is a project to update its language and tools while keeping proofs readable.**
- **I will give an overview. The handout keeps the details for later review.**
- The design is still open. I would welcome your comments and objections.

0.2 - A First Look

exact MML excerpt

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed view
  end;
end;
```

Mizar Evo

└ Part 0. Opening

└ 0.2 - A First Look

0.2 - A First Look

Current Mizar names the parent structure exact MML excerpt

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Evo also maps the fields to a common Magma view specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;    :: renamed view
  end;
end;
```

Read aloud:

- Current Mizar names the parent structure.
- Evo also maps the fields to a common Magma view.

Optional notes and sources:

- Both examples describe a carrier and a binary operation. Current Mizar already names the parent, 1-sorted. The Evo example adds a view of the same fields as a Magma.
- Source: current MML algstr_0.miz, lines 37-40.

0.3 - The Proposal In One Sentence

*Preserve Mizar's mathematical vernacular.
Update the compiler, verifier, artifact, and publication layers.*

This is the main proposal. We build on what current Mizar has achieved.

The language standard is still a draft. Full MML migration is not complete.

AI assistance must never replace proof checking.

The Evo examples follow the specification. Some are not yet implemented. I will explain the current implementation scope near the end.

Mizar Evo

└ Part 0. Opening

└ 0.3 - The Proposal In One Sentence

0.3 - The Proposal In One Sentence

Key Phrase:

*Preserve Mizar's mathematical vernacular.
Update the compiler, verifier, artifact, and publication layers.*

This is the main proposal. We build on what current Mizar has achieved.
The language standard is still a draft. Full MML migration is not complete.
AI assistance must never replace proof checking.
The Evo examples follow the specification. Some are not yet implemented. I will explain the current implementation scope near the end.

Read aloud:

- **Preserve Mizar's mathematical vernacular. Update the compiler, verifier, artifact, and publication layers.**
- This is the main proposal. We build on what current Mizar has achieved.
- **The language standard is still a draft. Full MML migration is not complete.**
- AI assistance must never replace proof checking.
- **The Evo examples follow the specification. Some are not yet implemented. I will explain the current implementation scope near the end.**

No EBNF appears in this talk. The language is shown through examples only; `doc/spec/en/` remains the authority for grammar and edge cases. Examples assume required imports and earlier declarations. . . . marks omitted proof text.

Mizar Evo

└ Part 0. Opening

└ 0.4 - How To Read The Examples

deep dive

Every code example is labeled:

Label	Meaning
exact MML excerpt	exact current MML text, with article and line numbers
specification example	from the Mizar Evo language specification
sketch	an example; not yet fixed by the specification

Reading rule:

No EBNF appears in this talk. The language is shown through examples only; `doc/spec/en/` remains the authority for grammar and edge cases. Examples assume required imports and earlier declarations. `...` marks omitted proof text.

Read aloud:

- No EBNF appears in this talk. The language is shown through examples only; `doc/spec/en/` remains the authority for grammar and edge cases.
- Examples assume required imports and earlier declarations. `...` marks omitted proof text.

0.5 - What We Need From This Visit

The purpose of this visit is to learn from your experience with Mizar.

I would like to identify compatibility needs and choose small MML articles for migration experiments.

I also hope to discuss how we check results from automated theorem provers, or ATPs. Another topic is how we link our work to Formalized Mathematics.

My main question is this.

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

2026-09-17

Mizar Evo

└ Part 0. Opening

└ 0.5 - What We Need From This Visit

0.5 - What We Need From This Visit

The purpose of this visit is to learn from your experience with Mizar.
I would like to identify compatibility needs and choose small MML articles for migration experiments.

I also hope to discuss how we check results from automated theorem provers, or ATPs. Another topic is how we link our work to Formalized Mathematics.
My main question is this.

Main question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Read aloud:

- **The purpose of this visit is to learn from your experience with Mizar.**
- I would like to identify compatibility needs and choose small MML articles for migration experiments.
- **I also hope to discuss how we check results from automated theorem provers, or ATPs. Another topic is how we link our work to Formalized Mathematics.**
- My main question is this.
- **What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?**

Optional notes and sources:

- There may be constraints that are hard to see without daily MML work. Please tell me about objections I have missed.

1.1 - What Mizar Got Right

First, let me explain what I want to keep.

Mizar proofs are declarative and read as mathematics.

Soft types, modes, and attributes give us a rich mathematical vocabulary.

Registrations and clusters provide automation that other proofs can reuse.

The Mizar Mathematical Library, or MML, is large and carefully maintained.

Formalized Mathematics provides a way to publish this work.

I want to judge each Evo proposal by how well it protects these strengths.

Mizar Evo

└ Part 1. Why Now

└ 1.1 - What Mizar Got Right

1.1 - What Mizar Got Right

First, let me explain what I want to keep.
Mizar proofs are declarative and read as mathematics.
Soft types, modes, and attributes give us a rich mathematical vocabulary.
Registrations and clusters provide automation that other proofs can reuse.
The Mizar Mathematical Library, or MML, is large and carefully maintained.
Formalized Mathematics provides a way to publish this work.
I want to judge each Evo proposal by how well it protects these strengths.

Read aloud:

- **First, let me explain what I want to keep.**
- **Mizar proofs are declarative and read as mathematics.**
- Soft types, modes, and attributes give us a rich mathematical vocabulary.
- **Registrations and clusters provide automation that other proofs can reuse.**
- **The Mizar Mathematical Library, or MML, is large and carefully maintained.**
- **Formalized Mathematics provides a way to publish this work.**
- I want to judge each Evo proposal by how well it protects these strengths.

Optional notes and sources:

- The library version cited here is MML 5.94.1493, with 1493 articles.

1.2 - Pressure One: Scale

The first reason for this work is library size.

MML has roughly 1500 articles that depend on each other.

An article is the unit of dependency, review, and reuse.

Tools resolve article environments, but the resolved dependencies are not visible in the source.

As the library grows, renaming and revising articles can affect more work. Evo asks whether smaller boundaries would help us maintain the library.

2026-09-17

Mizar Evo

└ Part 1. Why Now

└ 1.2 - Pressure One: Scale

1.2 - Pressure One: Scale

The first reason for this work is library size.
MML has roughly 1500 articles that depend on each other.
An article is the unit of dependency, review, and reuse.
Tools resolve article environments, but the resolved dependencies are not visible in the source.
As the library grows, renaming and revising articles can affect more work. Evo asks whether smaller boundaries would help us maintain the library.

Read aloud:

- **The first reason for this work is library size.**
- MML has roughly 1500 articles that depend on each other.
- **An article is the unit of dependency, review, and reuse.**
- Tools resolve article environments, but the resolved dependencies are not visible in the source.
- As the library grows, renaming and revising articles can affect more work. Evo asks whether smaller boundaries would help us maintain the library.

1.3 - Pressure Two: Tooling Expectations

The second reason is the way people now expect tools to work.

We expect editors to give quick feedback, even while the source is incomplete.

We expect a manifest and lockfile to make builds reproducible.

We expect packages with versions and documentation that we can browse.

These features are common in programming tools. I want them to support work on formal libraries too.

2026-09-17

Mizar Evo

└ Part 1. Why Now

└ 1.3 - Pressure Two: Tooling Expectations

1.3 - Pressure Two: Tooling Expectations

The second reason is the way people now expect tools to work.
We expect editors to give quick feedback, even while the source is incomplete.

We expect a manifest and lockfile to make builds reproducible.

We expect packages with versions and documentation that we can browse.

These features are common in programming tools. I want them to support work on formal libraries too.

Read aloud:

- **The second reason is the way people now expect tools to work.**
- **We expect editors to give quick feedback, even while the source is incomplete.**
- **We expect a manifest and lockfile to make builds reproducible.**
- We expect packages with versions and documentation that we can browse.
- These features are common in programming tools. I want them to support work on formal libraries too.

1.4 - Pressure Three: AI

The third reason is AI assistance.

AI tools can help us search, explain, and edit proofs.

They need a small amount of structured context linked to the source. They should not need a copy of the whole library for each task.

Their output must still be checked. Proof validity must not depend on how capable the AI tool is.

Mizar's readable source helps here. Stable local text patterns make editing and retrieval easier.

Mizar Evo

└ Part 1. Why Now

└ 1.4 - Pressure Three: AI

1.4 - Pressure Three: AI

The third reason is AI assistance.

AI tools can help us search, explain, and edit proofs.

They need a small amount of structured context linked to the source. They should not need a copy of the whole library for each task.

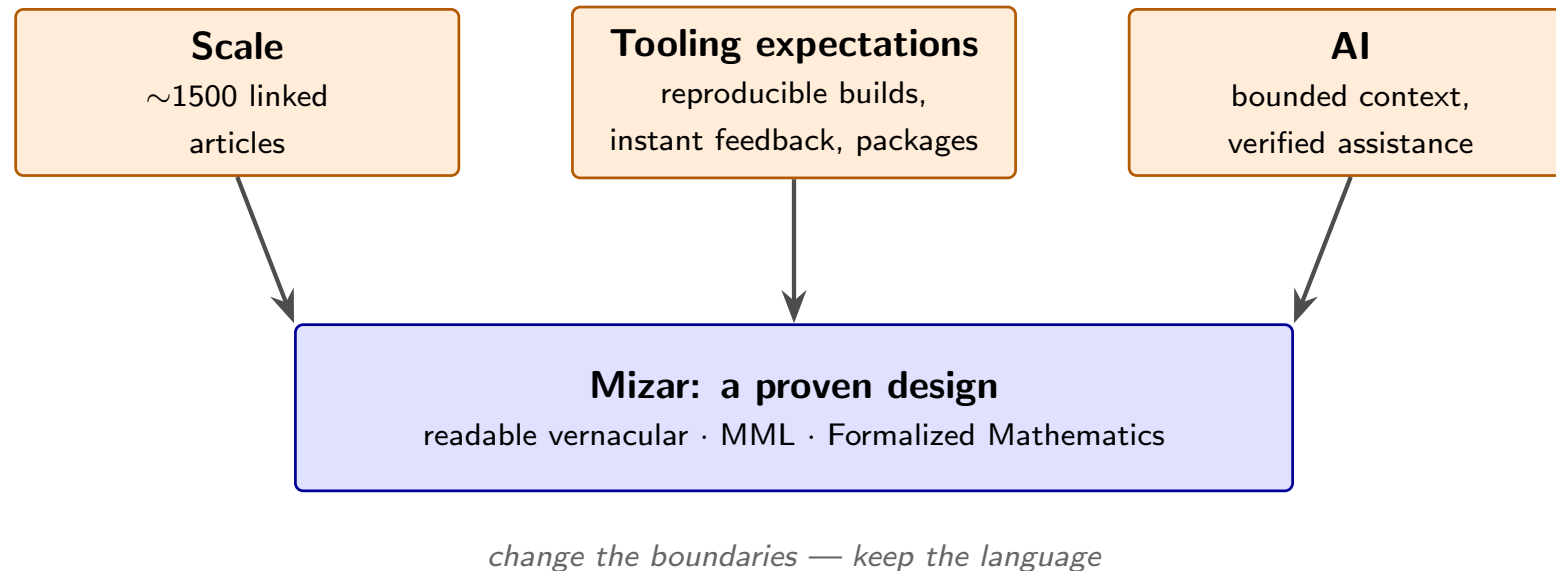
Their output must still be checked. Proof validity must not depend on how capable the AI tool is.

Mizar's readable source helps here. Stable local text patterns make editing and retrieval easier.

Read aloud:

- **The third reason is AI assistance.**
- **AI tools can help us search, explain, and edit proofs.**
- They need a small amount of structured context linked to the source. They should not need a copy of the whole library for each task.
- **Their output must still be checked. Proof validity must not depend on how capable the AI tool is.**
- Mizar's readable source helps here. Stable local text patterns make editing and retrieval easier.

1.5 - Three Pressures, One Design



This diagram brings the three reasons together: library size, tools, and AI.

They do not mean that Mizar's design was wrong.

They explain why I want to update some boundaries while keeping Mizar's readable mathematical language.

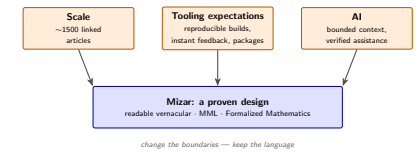
2026-09-17

Mizar Evo

└ Part 1. Why Now

└ 1.5 - Three Pressures, One Design

1.5 - Three Pressures, One Design



This diagram brings the three reasons together: library size, tools, and AI. They do not mean that Mizar's design was wrong. They explain why I want to update some boundaries while keeping Mizar's readable mathematical language.

Read aloud:

- This diagram brings the three reasons together: library size, tools, and AI.
- They do not mean that Mizar's design was wrong.
- They explain why I want to update some boundaries while keeping Mizar's readable mathematical language.

1.6 - The Design Rule

*Do not make proofs harder to read to add automation.
Use automation to protect and extend readability.*

This gives us three questions. Can people still read the proof? Can tools inspect its context? Will the design work as the library grows?

The next eight stories apply these questions to specific problems.

Mizar Evo

└ Part 1. Why Now

└ 1.6 - The Design Rule

1.6 - The Design Rule

Key Phrase:

*Do not make proofs harder to read to add automation.
Use automation to protect and extend readability.*

This gives us three questions. Can people still read the proof? Can tools inspect its context? Will the design work as the library grows?

The next eight stories apply these questions to specific problems.

Details for later review:

Goal	Test for every feature
Readability	does proof text still read as mathematics?
AI-readiness	can a tool see a limited context that we can audit?
Scalability	do boundaries stay stable as the library grows?

Read aloud:

- Do not make proofs harder to read to add automation. Use automation to protect and extend readability.
- This gives us three questions. Can people still read the proof? Can tools inspect its context? Will the design work as the library grows?
- The next eight stories apply these questions to specific problems.

2.1 - The Problem

exact MML excerpt

```
environ

vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,
             FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;
notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,
          CARD_1, STRUCT_0;
constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;
registrations ZFMISC_1, CARD_1, STRUCT_0;
theorems STRUCT_0;

begin :: Additive structures
```

Our first story is about dependencies. These lists tell Mizar which library material an article needs.

This form has supported library growth for decades. The question is how to make dependencies easier to review in a larger library.

Mizar Evo

└ Part 2. Story 1: Dependencies You Can See

└ 2.1 - The Problem

2.1 - The Problem

This environment lists dependencies [exact MML excerpt](#)

```
environ
vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,
  FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;
notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,
  CARD_1, STRUCT_0;
constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;
registrations ZFMISC_1, CARD_1, STRUCT_0;
theorems STRUCT_0;
begin :: Additive structures
```

Our first story is about dependencies. These lists tell Mizar which library material an article needs.
This form has supported library growth for decades. The question is how to make dependencies easier to review in a larger library.

Read aloud:

- This environment lists dependencies.
- **Our first story is about dependencies. These lists tell Mizar which library material an article needs.**
- **This form has supported library growth for decades. The question is how to make dependencies easier to review in a larger library.**

Optional notes and sources:

- Source: current MML algstr_0.miz, lines 15-25.

Article order matters in environment lists such as notations and definitions. Authors must understand not only which articles to include, but also how to order them.

The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.

Users must list individual articles even when they want a whole topic, such as elementary group theory.

Moving a theorem between articles risks breaking unknown dependents.

Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

Mizar Evo

└ Part 2. Story 1: Dependencies You Can See

└ 2.2 - Why This Is Difficult

deep dive

2.2 - Why This Is Difficult

deep dive

Key Points:

Article order matters in environment lists such as notations and definitions. Authors must understand not only which articles to include, but also how to order them.

The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.

Users must list individual articles even when they want a whole topic, such as elementary group theory.

Moving a theorem between articles risks breaking unknown dependents.

Message:

Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

Read aloud:

- Article order matters in environment lists such as notations and definitions. Authors must understand not only which articles to include, but also how to order them.
- The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.
- Users must list individual articles even when they want a whole topic, such as elementary group theory.
- Moving a theorem between articles risks breaking unknown dependents.
- Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

2.3 - The Evo Answer: Import Prelude

Evo puts imports before definitions specification example

```
import .function;  
import mml.algebra.structure.sorted;  
  
definition  
  let S be 1-sorted;  
  mode BinOpDef: BinOp of S is  
    Function of [: S.carrier, S.carrier :], S.carrier;  
end;
```

Here, the imports come before the first non-import item.

They provide the initial active lexicon. Local declarations extend it after their declaration points.

Imported items keep their source FQNs. Package and module paths give each item a stable fully-qualified name.

2026-09-17

Mizar Evo

└ Part 2. Story 1: Dependencies You Can See

└ 2.3 - The Evo Answer: Import Prelude

2.3 - The Evo Answer: Import Prelude

Evo puts imports before definitions specification example

```
import .function;  
import mml.algebra.structure.sorted;  
  
definition  
  let S be 1-sorted;  
  mode BinOpDef: BinOp of S is  
    Function of [: S.carrier, S.carrier :], S.carrier;  
end;
```

Here, the imports come before the first non-import item.
They provide the initial active lexicon. Local declarations extend it after their declaration points.
Imported items keep their source FQNs. Package and module paths give each item a stable fully-qualified name.

Read aloud:

- **Evo puts imports before definitions.**
- **Here, the imports come before the first non-import item.**
- **They provide the initial active lexicon. Local declarations extend it after their declaration points.**
- **Imported items keep their source FQNs. Package and module paths give each item a stable fully-qualified name.**

specification example

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

With `export`, a library can offer a topic such as elementary group theory through one aggregate module and a single import.

reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence;

versioned reuse (SemVer) replaces manual copying between article sets.

2026-09-17

Mizar Evo

└ Part 2. Story 1: Dependencies You Can See

└ 2.4 - The Evo Answer: Packages

deep dive

2.4 - The Evo Answer: Packages

deep dive

Mizar Evo **specification example**

```
[package]
name    = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "~1.0"
topology = { version = "~0.9", features = ["metric"] }
```

Key Points:

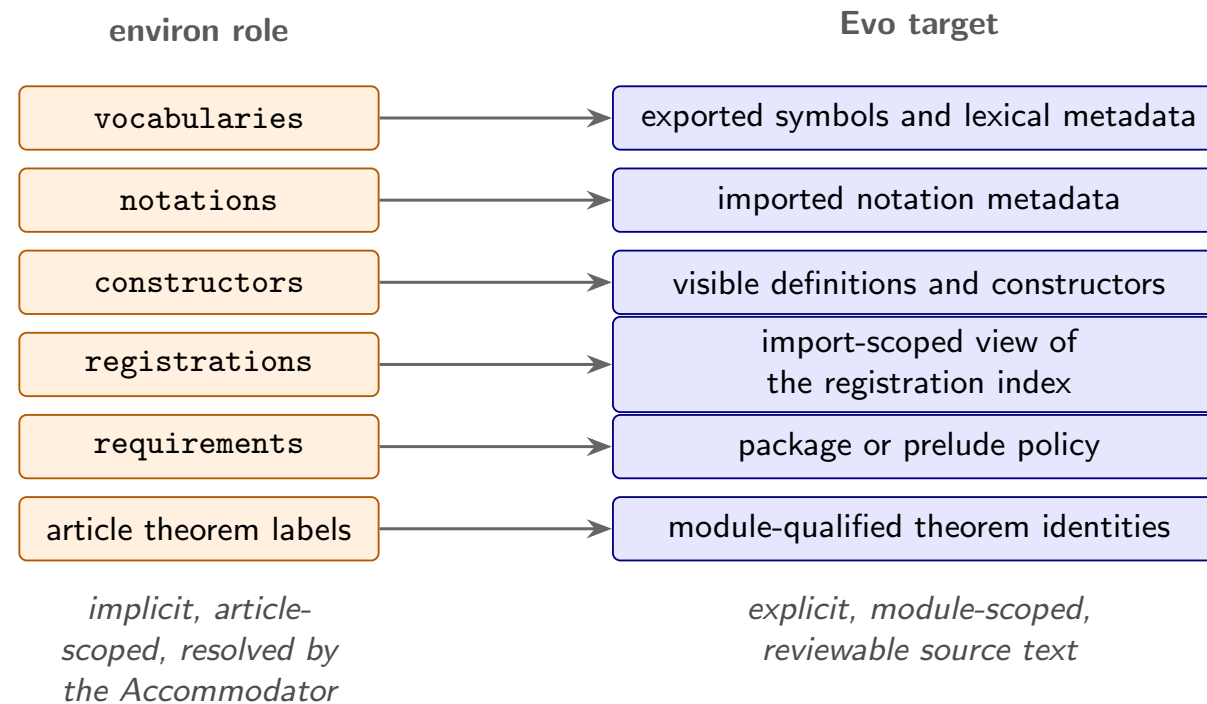
With export, a library can offer a topic such as elementary group theory through one aggregate module and a single import.
reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence;
versioned reuse (SemVer) replaces manual copying between article sets.

Read aloud:

- With export, a library can offer a topic such as elementary group theory through one aggregate module and a single import.
- reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence;
- versioned reuse (SemVer) replaces manual copying between article sets.

2.5 - Migrating The Environment

deep dive



Migration changes more than names: reports must show each module's syntax, semantics, and automation.

2026-09-17

Mizar Evo

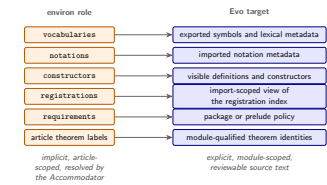
└ Part 2. Story 1: Dependencies You Can See

└ 2.5 - Migrating The Environment

deep dive

2.5 - Migrating The Environment

deep dive



Message:

Migration changes more than names: reports must show each module's syntax, semantics, and automation.

Read aloud:

- Migration changes more than names: reports must show each module's syntax, semantics, and automation.

2.6 - What Is Preserved, What We Ask

The aim is to preserve the mathematics and theorem identities during migration.

A module is still a readable text, with article-style authorship.

Origin metadata records where migrated items came from. I will return to this when I discuss publication.

The questions below are for later review. I will now turn to structures.

Which `environ` roles must remain visually familiar during migration?

Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

Mizar Evo

└ Part 2. Story 1: Dependencies You Can See

└ 2.6 - What Is Preserved, What We Ask

2.6 - What Is Preserved, What We Ask

The aim is to preserve the mathematics and theorem identities during migration.

A module is still a readable text, with article-style authorship.

Origin metadata records where migrated items came from. I will return to this when I discuss publication.

The questions below are for later review. I will now turn to structures.

Questions for later review:

Which environ roles must remain visually familiar during migration?

Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

Read aloud:

- **The aim is to preserve the mathematics and theorem identities during migration.**
- A module is still a readable text, with article-style authorship.
- Origin metadata records where migrated items came from. I will return to this when I discuss publication.
- **The questions below are for later review. I will now turn to structures.**

3.1 - The Problem

exact MML excerpt

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

One declaration contains the parent link, fields, and selectors.

With several parents, it also determines which inherited fields are shared.

Additive and multiplicative views depend on naming conventions.

The syntax does not separate stored data from canonical values such as zero.

Evo proposes a more explicit way to state these choices.

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.1 - The Problem

3.1 - The Problem

This structure uses the familiar compact form [exact MML excerpt](#)

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

One declaration contains the parent link, fields, and selectors.

With several parents, it also determines which inherited fields are shared.

Additive and multiplicative views depend on naming conventions.

The syntax does not separate stored data from canonical values such as zero.

Evo proposes a more explicit way to state these choices.

Read aloud:

- This structure uses the familiar compact form.
- **One declaration contains the parent link, fields, and selectors.**
- With several parents, it also determines which inherited fields are shared.
- Additive and multiplicative views depend on naming conventions.
- **The syntax does not separate stored data from canonical values such as zero.**
- Evo proposes a more explicit way to state these choices.

Optional notes and sources:

- This short form keeps structures close to informal mathematical writing.
- Source: current MML algstr_0.miz, lines 37-40.

Current Mizar defines `addMagma` and `multMagma` separately, although both describe a carrier with one binary operation.

Common definitions and proofs are repeated in additive and multiplicative forms.

Evo's member renaming lets both structures use a common `Magma` theory, while keeping their familiar operation names.

Define and prove the common theory once, then reuse it through renamed views.

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.2 - Why This Is Difficult

deep dive

3.2 - Why This Is Difficult

deep dive

Key Points:

Current Mizar defines `addMagma` and `multMagma` separately, although both describe a carrier with one binary operation.
Common definitions and proofs are repeated in additive and multiplicative forms.
Evo's member renaming lets both structures use a common `Magma` theory, while keeping their familiar operation names.

Message:

Define and prove the common theory once, then reuse it through renamed views.

Read aloud:

- Current Mizar defines `addMagma` and `multMagma` separately, although both describe a carrier with one binary operation.
- Common definitions and proofs are repeated in additive and multiplicative forms.
- Evo's member renaming lets both structures use a common `Magma` theory, while keeping their familiar operation names.
- **Define and prove the common theory once, then reuse it through renamed views.**

3.3 - The Evo Answer: Field, Property, Attribute

Evo separates fields, properties, and attributes specification example

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

A field stores data, like carrier and add here. Fields are constructor arguments and determine equality of exact instances.

A property, like zero here, gets its value from an implementation. The declaration alone supplies no value. means requires existence and uniqueness; equals gives a term.

An attribute is a predicate-style refinement. It supports cluster propagation and does not change the layout.

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.3 - The Evo Answer: Field, Property, Attribute

3.3 - The Evo Answer: Field, Property, Attribute

Evo separates fields, properties, and attributes specification example

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
end;
end;
```

A field stores data, like `carrier` and `add` here. Fields are constructor arguments and determine equality of exact instances.

A property, like `zero` here, gets its value from an implementation. The declaration alone supplies no value. `means` requires existence and uniqueness; `equals` gives a term.

An attribute is a predicate-style refinement. It supports cluster propagation and does not change the layout.

Read aloud:

- **Evo separates fields, properties, and attributes.**
- **A field stores data, like `carrier` and `add` here. Fields are constructor arguments and determine equality of exact instances.**
- **A property, like `zero` here, gets its value from an implementation. The declaration alone supplies no value. `means` requires existence and uniqueness; `equals` gives a term.**
- **An attribute is a predicate-style refinement. It supports cluster propagation and does not change the layout.**

Optional notes and sources:

- Overlapping property implementations need coherence.

3.4 - The Evo Answer: Explicit Inheritance

Evo writes each parent mapping explicitly specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed
  end
end
```

There is one `inherit` statement for each parent.

Here, `from` maps the child's `add` field to the parent's `binop` field.

Syntactically identical member types need no proof, even if a name changes.

Other types need coherence proofs of subtype inclusion.

2026-09-17

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.4 - The Evo Answer: Explicit Inheritance

3.4 - The Evo Answer: Explicit Inheritance

Evo writes each parent mapping explicitly specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;    :: renamed
  end;
end;
```

There is one inherit statement for each parent.

Here, from maps the child's add field to the parent's binop field.

Syntactically identical member types need no proof, even if a name changes.

Other types need coherence proofs of subtype inclusion.

Read aloud:

- **Evo writes each parent mapping explicitly.**
- **There is one inherit statement for each parent.**
- **Here, from maps the child's add field to the parent's binop field.**
- **Syntactically identical member types need no proof, even if a name changes.**
Other types need coherence proofs of subtype inclusion.

3.5 - The Evo Answer: Diamonds Become Checkable (1/2)

specification example

```
definition
  struct DoubleLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    field mul -> BinOp of carrier;
    property zero -> Element of carrier;
    property one -> Element of carrier;
  end

  inherit DoubleLoopStr extends AddLoopStr;
  inherit DoubleLoopStr extends MulLoopStr;
end
```

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.5 - The Evo Answer: Diamonds Become Checkable (1/2)

3.5 - The Evo Answer: Diamonds Become Checkable (1/2)

This child has two parent structures specification example

```
definition
  struct DoubleLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    field mul -> BinOp of carrier;
    property zero -> Element of carrier;
    property one -> Element of carrier;
  end;

  inherit DoubleLoopStr extends AddLoopStr;
  inherit DoubleLoopStr extends MulLoopStr;
end;
```

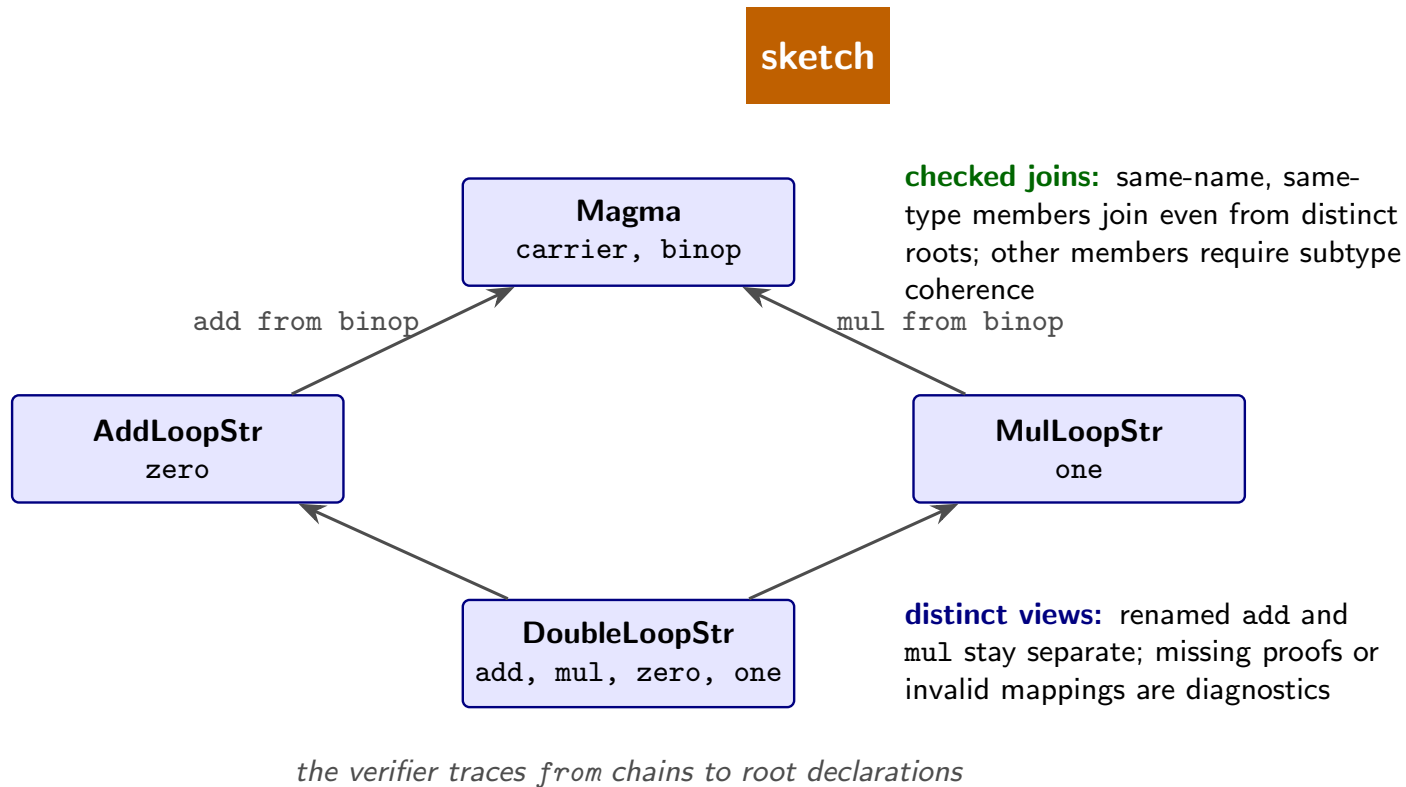
Read aloud:

- This child has two parent structures.

Optional notes and sources:

- The diagram assumes the shown mappings from AddLoopStr and MulLoopStr to Magma. The code adds their shared child, DoubleLoopStr.
- Same-name, same-type members can join across roots. Different member types need coherence proofs of subtype inclusion.

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)



Define and prove once for Magma; reuse through the additive and multiplicative views.

The views stay distinct; theorem assumptions must hold for each view.

2026-09-17

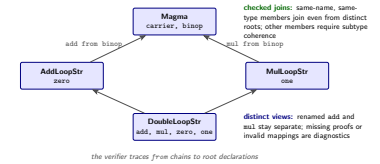
Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.5 - The Evo Answer: Diamonds Become Checkable (2/2)

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)

The diagram shows the inheritance paths sketch



Define and prove once for Magma; reuse through the additive and multiplicative views.

The views stay distinct; theorem assumptions must hold for each view.

Read aloud:

- The diagram shows the inheritance paths.
- **Define and prove once for Magma; reuse through the additive and multiplicative views.**
- The views stay distinct; theorem assumptions must hold for each view.

3.6 - What Is Preserved, What We Ask

We still use carriers, selectors, and `Element of`, as in Mizar.

Aggregates become longer only when we need to make a hidden choice explicit.

The main question is whether this form is readable in real algebraic articles. I would like to test it on structures with several inheritance paths. Next, I will discuss registrations and clusters.

Is the `field / property / attribute` split readable in real algebraic articles, or does it require too many annotations in simple cases?

Is `one-parent-per-inherit` acceptable for parts of MML with much inheritance, such as the `ALGSTR` and topology hierarchies?

Which MML structures would be the best diamond test cases?

2026-09-17

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.6 - What Is Preserved, What We Ask

3.6 - What Is Preserved, What We Ask

We still use carriers, selectors, and `Element of`, as in Mizar.

Aggregates become longer only when we need to make a hidden choice explicit.

The main question is whether this form is readable in real algebraic articles.

I would like to test it on structures with several inheritance paths. Next, I will discuss registrations and clusters.

Questions for later review:

Is the `field / property / attribute split` readable in real algebraic articles, or does it require too many annotations in simple cases?

Is `one-parent-per-inherit` acceptable for parts of MML with much inheritance, such as the ALGSTR and topology hierarchies?

Which MML structures would be the best diamond test cases?

Read aloud:

- We still use carriers, selectors, and `Element of`, as in Mizar.
- Aggregates become longer only when we need to make a hidden choice explicit.
- **The main question is whether this form is readable in real algebraic articles.**
- **I would like to test it on structures with several inheritance paths. Next, I will discuss registrations and clusters.**

4.1 - The Problem

exact MML excerpt

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
    Element
      of M;
  coherence;
end;
```

**Registrations let attributes propagate automatically, so proofs stay short.
I want to keep this idea and make the steps of the automation easier to see.**

Mizar Evo

└ Part 4. Story 3: Automation You Can Audit

└ 4.1 - The Problem

4.1 - The Problem

This registration lets Mizar reuse a proved fact [exact MML excerpt](#)

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
    Element
      of M;
  coherence;
end;
```

Registrations let attributes propagate automatically, so proofs stay short.
I want to keep this idea and make the steps of the automation easier to see.

Read aloud:

- This registration lets Mizar reuse a proved fact.
- **Registrations let attributes propagate automatically, so proofs stay short.**
- **I want to keep this idea and make the steps of the automation easier to see.**

Optional notes and sources:

- Source: current MML algstr_0.miz, lines 104-109.

When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.

Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.

For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Automation without explanations makes a large library harder to maintain, even when it is sound.

Mizar Evo

└ Part 4. Story 3: Automation You Can Audit

└ 4.2 - Why This Is Difficult

deep dive

4.2 - Why This Is Difficult

deep dive

Key Points:

When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.
Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.
For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:

Automation without explanations makes a large library harder to maintain, even when it is sound.

Read aloud:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.
- Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.
- Automation without explanations makes a large library harder to maintain, even when it is sound.

4.3 - The Evo Answer: Labeled, Traceable Registrations

Evo gives every registration item a label **specification example**

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Labels make cluster use traceable. Explicit citation in by is optional.

The verifier applies applicable clusters automatically within the active environment.

explain-attribute and resolution traces explain success or failure. For example, they can show the steps from empty to finite to countable.

Mizar Evo

└ Part 4. Story 3: Automation You Can Audit

└ 4.3 - The Evo Answer: Labeled, Traceable Registrations

4.3 - The Evo Answer: Labeled, Traceable Registrations

Evo gives every registration item a label specification example

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Labels make cluster use traceable. Explicit citation in `by` is optional. The verifier applies applicable clusters automatically within the active environment. `explain-attribute` and resolution traces explain success or failure. For example, they can show the steps from empty to finite to countable.

Read aloud:

- Evo gives every registration item a label.
- Labels make cluster use traceable. Explicit citation in `by` is optional.
- The verifier applies applicable clusters automatically within the active environment.
- `explain-attribute` and resolution traces explain success or failure. For example, they can show the steps from empty to finite to countable.

specification example

```
registration
  let n be Nat;
  reduce NatAddZero: n + 0 to n;
  reducibility
  proof
    let n be Nat;
    thus n + 0 = n by mml.number.natural.Nat_add_zero;
  end
end;
```

a reduction is an oriented simplification backed by an equality proof;
the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
unoriented identification idioms become auditable reduce items.

2026-09-17

Mizar Evo

└ Part 4. Story 3: Automation You Can Audit

└ 4.4 - The Evo Answer: Oriented Reductions

deep dive

4.4 - The Evo Answer: Oriented Reductions

deep dive

Mizar Evo **specification example**

```
registration
  let n be Nat;
  reduce NatAddZero: n + 0 to n;
  reducibility
  proof
    let n be Nat;
    thus n + 0 = n by mml.number.natural.Nat_add_zero;
  end;
end;
```

Key Points:

a reduction is an oriented simplification backed by an equality proof;
the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
unoriented identification idioms become auditable `reduce` items.

Read aloud:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable `reduce` items.

4.5 - What Is Preserved, What We Ask

Registrations and clusters remain part of the language, and proofs stay short.

Applying a cluster does not require us to repeat its proof.

A reduction simplifies a term using an equality. It may need local guard evidence or an explicit equality citation.

**I would like to learn which explanations would help most in daily MML work.
This leads to the next story: stronger proof search.**

Which cluster explanations would help most with daily work: failure explanations, firing traces, or difference reports between environments?

Which MML article families make the most complex use of registrations and should become migration benchmarks for the cluster graph?

Mizar Evo

└ Part 4. Story 3: Automation You Can Audit

└ 4.5 - What Is Preserved, What We Ask

4.5 - What Is Preserved, What We Ask

Registrations and clusters remain part of the language, and proofs stay short.

Applying a cluster does not require us to repeat its proof.

A reduction simplifies a term using an equality. It may need local guard evidence or an explicit equality citation.

I would like to learn which explanations would help most in daily MML work. This leads to the next story: stronger proof search.

Questions for later review:

Which cluster explanations would help most with daily work: failure explanations, firing traces, or difference reports between environments?

Which MML article families make the most complex use of registrations and should become migration benchmarks for the cluster graph?

Read aloud:

- Registrations and clusters remain part of the language, and proofs stay short.
- Applying a cluster does not require us to repeat its proof.
- A reduction simplifies a term using an equality. It may need local guard evidence or an explicit equality citation.
- **I would like to learn which explanations would help most in daily MML work. This leads to the next story: stronger proof search.**

5.1 - The Problem

Users want stronger automation, including larger by steps and hammer-style search.

MizAR and MPTP already use ATP search with MML premises.

Evo's goal is to keep proof checking small as search becomes stronger.

A search result alone is not a kernel-verified proof. We need checkable evidence.

So the question is this.

*How do we get modern proof search
without trusting the searcher?*

Mizar Evo

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.1 - The Problem

5.1 - The Problem

Users want stronger automation, including larger by steps and hammer-style search.
Mizar and MPTP already use ATP search with MML premises.
Evo's goal is to keep proof checking small as search becomes stronger.
A search result alone is not a kernel-verified proof. We need checkable evidence.
So the question is this.

Key Phrase:

*How do we get modern proof search
without trusting the searcher?*

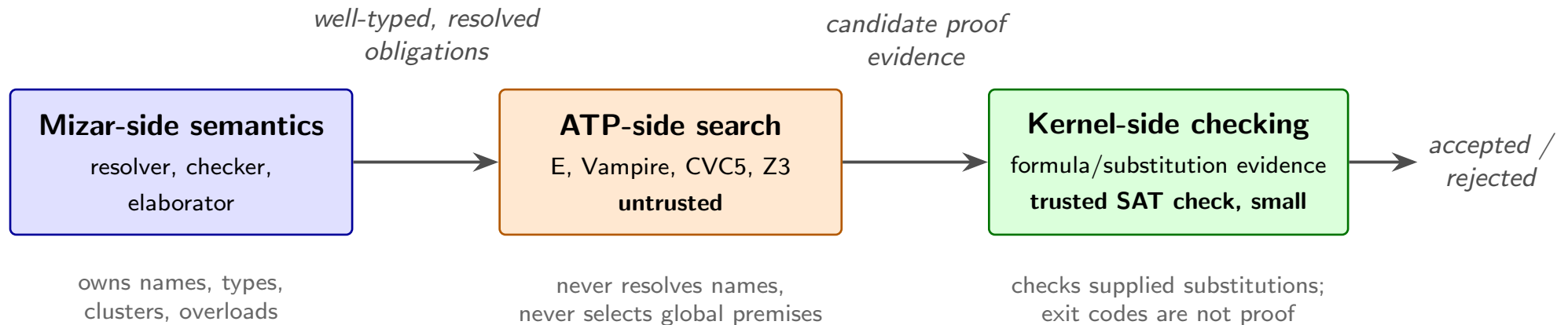
Read aloud:

- Users want stronger automation, including larger by steps and hammer-style search.
- **Mizar and MPTP already use ATP search with MML premises.**
- **Evo's goal is to keep proof checking small as search becomes stronger.**
- **A search result alone is not a kernel-verified proof. We need checkable evidence.**
- So the question is this.
- How do we get modern proof search without trusting the searcher?

Optional notes and sources:

- This builds on Mizar, MPTP, and hammer research. I am describing Evo's evidence contract, not making a claim that earlier tools must trust their search backends.

5.2 - The Evo Answer: A Reasoning Boundary



We can read this diagram from left to right.

Mizar-side phases resolve names, infer types, expand clusters, and pick overloads. ATPs do none of these tasks.

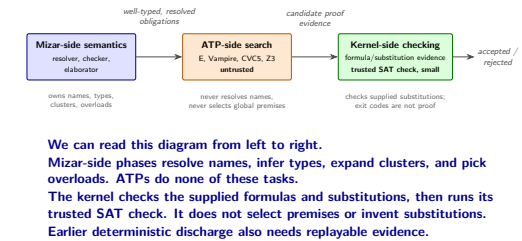
The kernel checks the supplied formulas and substitutions, then runs its trusted SAT check. It does not select premises or invent substitutions. Earlier deterministic discharge also needs replayable evidence.

Mizar Evo

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.2 - The Evo Answer: A Reasoning Boundary

5.2 - The Evo Answer: A Reasoning Boundary



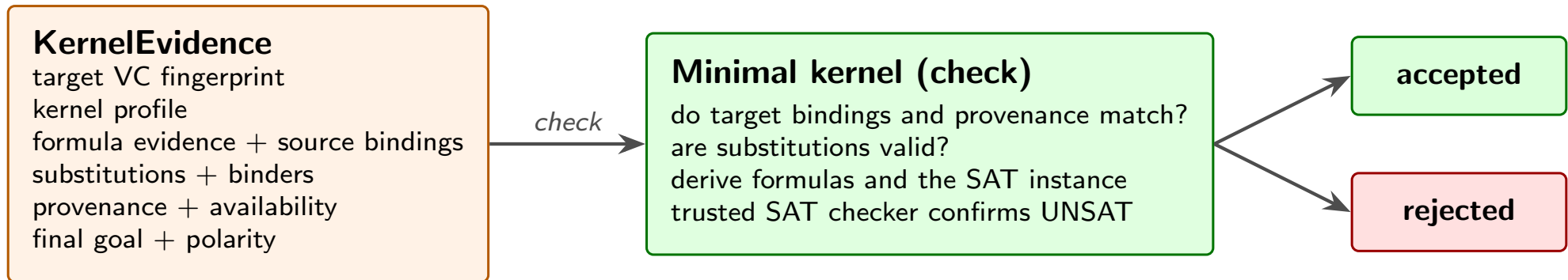
Read aloud:

- We can read this diagram from left to right.
- Mizar-side phases resolve names, infer types, expand clusters, and pick overloads. ATPs do none of these tasks.
- The kernel checks the supplied formulas and substitutions, then runs its trusted SAT check. It does not select premises or invent substitutions.
- Earlier deterministic discharge also needs replayable evidence.

Optional notes and sources:

- Development policy may record externally_attested results. These are separate from kernel-verified proofs.

5.3 - Formula And Substitution Evidence



*solver exit codes and backend traces are diagnostic only —
hashes link evidence to its dependency slice*

Here we can see what the evidence contains: source formulas, substitutions, provenance, and target and goal bindings.

The kernel checks it, derives instantiated formulas and SAT clauses, and requires UNSAT from its trusted in-process Rust SAT checker.

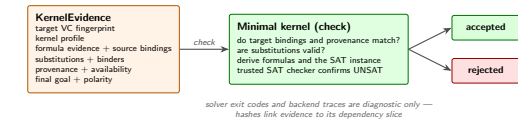
Backend resolution traces, SMT proof objects, logs, and exit codes are not trusted acceptance evidence.

Mizar Evo

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.3 - Formula And Substitution Evidence

5.3 - Formula And Substitution Evidence



Here we can see what the evidence contains: source formulas, substitutions, provenance, and target and goal bindings.
The kernel checks it, derives instantiated formulas and SAT clauses, and requires UNSAT from its trusted in-process Rust SAT checker.
Backend resolution traces, SMT proof objects, logs, and exit codes are not trusted acceptance evidence.

Read aloud:

- Here we can see what the evidence contains: source formulas, substitutions, provenance, and target and goal bindings.
- The kernel checks it, derives instantiated formulas and SAT clauses, and requires UNSAT from its trusted in-process Rust SAT checker.
- Backend resolution traces, SMT proof objects, logs, and exit codes are not trusted acceptance evidence.

Optional notes and sources:

- Hashes bind evidence to its dependency context. The kernel also checks binder conditions and the goal's refutation polarity.

5.4 - The Same Boundary Controls AI

exact MML excerpt

```
theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;
```

The AI tool can propose a missing or more precise by reference.

This is a local edit that keeps the statement's meaning.

The verifier checks it just as it would check a human edit.

The AI proposes the change. The verifier and kernel decide whether to accept it.

Mizar Evo

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.4 - The Same Boundary Controls AI

5.4 - The Same Boundary Controls AI

An AI tool could help repair a citation like this one exact MML excerpt

```

theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;

```

The AI tool can propose a missing or more precise by reference.

This is a local edit that keeps the statement's meaning.

The verifier checks it just as it would check a human edit.

The AI proposes the change. The verifier and kernel decide whether to accept it.

Read aloud:

- An AI tool could help repair a citation like this one.
- **The AI tool can propose a missing or more precise by reference.**
- This is a local edit that keeps the statement's meaning.
- **The verifier checks it just as it would check a human edit.**
- The AI proposes the change. The verifier and kernel decide whether to accept it.

Optional notes and sources:

- The assistant's strength never enters the trusted base.
- Source: current MML struct_0.miz, lines 637-643.

sketch

```
theorem  
  for x be Nat holds x + 0 = x or x = 0;
```

Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

2026-09-17

Mizar Evo

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.5 - Edit Classes: Green, Yellow, Red

deep dive

5.5 - Edit Classes: Green, Yellow, Red

deep dive

Class	Examples	Policy
Green	add citation, insert qua, info annotation	can be proposed automatically; still verified
Yellow	add import, local lemma, registration	proposed with human review
Red	weaken theorem, change definition, add axiom	forbidden to ordinary agents

Forbidden repair sketch

theorem

for x be Nat holds x + 0 = x or x = 0;

Message:

Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

Read aloud:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

5.6 - What Is Preserved, What We Ask

This keeps the de Bruijn discipline: a small trusted core checks acceptance. In this design, the core includes its SAT checker.

Proof text stays declarative and readable. Automation helps maintain the argument. I would welcome your views on the evidence format.

Now let us look at the cost of checking a large library.

Is the formula/substitution evidence approach convincing for Mizar-style obligations, including clusters and definitional expansions?

Which evidence format would the team be most willing to audit?

2026-09-17

Mizar Evo

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.6 - What Is Preserved, What We Ask

5.6 - What Is Preserved, What We Ask

This keeps the de Bruijn discipline: a small trusted core checks acceptance. In this design, the core includes its SAT checker.
Proof text stays declarative and readable. Automation helps maintain the argument.
I would welcome your views on the evidence format.

Now let us look at the cost of checking a large library.

Questions for later review:

Is the formula/substitution evidence approach convincing for Mizar-style obligations, including clusters and definitional expansions?
Which evidence format would the team be most willing to audit?

Read aloud:

- This keeps the de Bruijn discipline: a small trusted core checks acceptance. In this design, the core includes its SAT checker.
- Proof text stays declarative and readable. Automation helps maintain the argument.
- I would welcome your views on the evidence format.
- **Now let us look at the cost of checking a large library.**

6.1 - The Problem

Checking the whole MML takes hours.

The accepted article is the unit of reuse. A small edit can therefore cause more checking than we would like.

Memory use follows the article environment, including material outside the interface actually used.

These costs follow from the article as a unit. Evo proposes smaller units of reuse.

2026-09-17

Mizar Evo

└ Part 6. Story 5: Verification That Scales

└ 6.1 - The Problem

6.1 - The Problem

Checking the whole MML takes hours.

The accepted article is the unit of reuse. A small edit can therefore cause more checking than we would like.

Memory use follows the article environment, including material outside the interface actually used.

These costs follow from the article as a unit. Evo proposes smaller units of reuse.

Read aloud:

- **Checking the whole MML takes hours.**
- **The accepted article is the unit of reuse. A small edit can therefore cause more checking than we would like.**
- Memory use follows the article environment, including material outside the interface actually used.
- These costs follow from the article as a unit. Evo proposes smaller units of reuse.

6.2 - The Evo Answer: Fingerprints And Incrementality (1/2)

Evo uses fingerprints to decide when earlier results can be reused.

All relevant cache keys must match. Missing data causes a cache miss.

A proof-body edit does not rebuild importers when the exported statement and accepted status stay unchanged.

Independent modules, obligations, ATP runs, and kernel checks can run in parallel. Results are published in canonical order.

Mizar Evo

└ Part 6. Story 5: Verification That Scales

└ 6.2 - The Evo Answer: Fingerprints And Incrementality (1/2)

6.2 - The Evo Answer: Fingerprints And Incrementality (1/2)

Evo uses fingerprints to decide when earlier results can be reused.
All relevant cache keys must match. Missing data causes a cache miss.
A proof-body edit does not rebuild importers when the exported statement and accepted status stay unchanged.
Independent modules, obligations, ATP runs, and kernel checks can run in parallel.
Results are published in canonical order.

Read aloud:

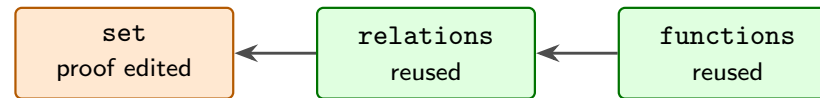
- **Evo uses fingerprints to decide when earlier results can be reused.**
- **All relevant cache keys must match. Missing data causes a cache miss.**
- **A proof-body edit does not rebuild importers when the exported statement and accepted status stay unchanged.**
- Independent modules, obligations, ATP runs, and kernel checks can run in parallel.
Results are published in canonical order.

Optional notes and sources:

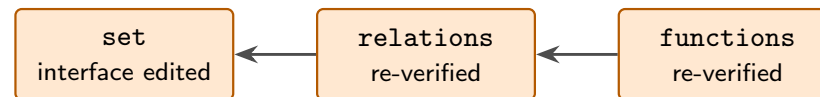
- The cache keys cover source, dependency slices, package and lockfile, toolchain, schema, policy, registrations, obligations, evidence, and witnesses.

6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

***Cache reuse is never proof authority.
A clean build must reproduce every acceptance.***

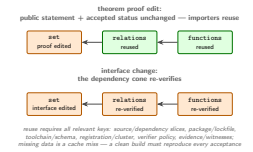
2026-09-17

Mizar Evo

└ Part 6. Story 5: Verification That Scales

└ 6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)

6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)



*Cache reuse is never proof authority.
A clean build must reproduce every acceptance.*

Read aloud:

- **Cache reuse is never proof authority. A clean build must reproduce every acceptance.**

6.3 - The Evo Answer: A Memory Contract

deep dive

```
resident memory should scale with:  
  active source and typed AST  
  imported public interfaces  
  import-filtered indexes  
  active module obligations  
  in-flight proof checks and ATP runs  
  small caches  
  
not with:  
  imported proof bodies  
  imported proof witnesses  
  private lemmas outside the interface  
  registration data outside the import closure
```

This is a resident-memory model, not a measured performance guarantee. Proof bodies and traces load lazily for specific queries.

2026-09-17

Mizar Evo

└ Part 6. Story 5: Verification That Scales

└ 6.3 - The Evo Answer: A Memory Contract

deep dive

6.3 - The Evo Answer: A Memory Contract

deep dive

```
resident memory should scale with:  
  active source and typed AST  
  imported public interfaces  
  import-filtered indexes  
  active module obligations  
  in-flight proof checks and ATP runs  
  small caches  
  
not with:  
  imported proof bodies  
  imported proof witnesses  
  private lemmas outside the interface  
  registration data outside the import closure
```

Message:

This is a resident-memory model, not a measured performance guarantee. Proof bodies and traces load lazily for specific queries.

Read aloud:

- This is a resident-memory model, not a measured performance guarantee. Proof bodies and traces load lazily for specific queries.

6.4 - What Is Preserved, What We Ask

Caching and parallel work change checking time. They must not change truth.

A person still reviews complete source text, in the style of an article.

We need tests that compare incremental results with a clean build.

I would like to use real MML maintenance tasks for these tests. Next, I will turn to templates.

What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?

Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

2026-09-17

Mizar Evo

└ Part 6. Story 5: Verification That Scales

└ 6.4 - What Is Preserved, What We Ask

6.4 - What Is Preserved, What We Ask

Caching and parallel work change checking time. They must not change truth.
A person still reviews complete source text, in the style of an article.

We need tests that compare incremental results with a clean build.

I would like to use real MML maintenance tasks for these tests. Next, I will turn to templates.

Questions for later review:

What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?

Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

Read aloud:

- Caching and parallel work change checking time. They must not change truth.
- A person still reviews complete source text, in the style of an article.
- **We need tests that compare incremental results with a clean build.**
- **I would like to use real MML maintenance tasks for these tests. Next, I will turn to templates.**

7.1 - The Problem, Part One: Schemes Use Separate Rules

This familiar scheme expresses induction exact MML excerpt

```
scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
A1: P[0] and
A2: for k be Nat st P[k] holds P[k + 1]
```

Schemes support second-order patterns such as induction, separation, and replacement.

Classic scheme blocks define theorem schemas. Parameterized definitions use other language forms.

Evo proposes a common template system for these tasks.

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.1 - The Problem, Part One: Schemes Use Separate Rules

7.1 - The Problem, Part One: Schemes Use Separate Rules

This familiar scheme expresses induction [exact MML excerpt](#)

```

scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
  A1: P[0] and
  A2: for k be Nat at P[k] holds P[k + 1]

```

Schemes support second-order patterns such as induction, separation, and replacement.

Classic scheme blocks define theorem schemas. Parameterized definitions use other language forms.

Evo proposes a common template system for these tasks.

Read aloud:

- **This familiar scheme expresses induction.**
- Schemes support second-order patterns such as induction, separation, and replacement.
- **Classic scheme blocks define theorem schemas. Parameterized definitions use other language forms.**
- **Evo proposes a common template system for these tasks.**

Optional notes and sources:

- Source: current MML nat_1.miz, near line 90 (checked July 2, 2026).

7.2 - The Problem, Part Two: A Common Template System

Mizar already has parameterized constructions, such as Polynom-Ring L.

The aim is to give definitions and theorem schemas a common template system.

Each kind of parameter still has explicit rules.

We need migration examples to see whether these rules make generic mathematics easier to write and maintain.

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.2 - The Problem, Part Two: A Common Template System

7.2 - The Problem, Part Two: A Common Template System

Mizar already has parameterized constructions, such as Polynom-Ring L.
The aim is to give definitions and theorem schemas a common template system.
Each kind of parameter still has explicit rules.
We need migration examples to see whether these rules make generic mathematics easier to write and maintain.
Source: POLYNOM3, definition 10
(<https://mizar.uwb.edu.pl/version/current/html/polynom3.html>).

Read aloud:

- **Mizar already has parameterized constructions, such as Polynom-Ring L.**
- The aim is to give definitions and theorem schemas a common template system.
- Each kind of parameter still has explicit rules.
- We need migration examples to see whether these rules make generic mathematics easier to write and maintain.

Optional notes and sources:

- Source: POLYNOM3, definition 10
(<https://mizar.uwb.edu.pl/version/current/html/polynom3.html>).

7.3 - The Evo Answer: Templates

This template begins with a type parameter **specification example**

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

A template is an ordinary definition block. Its leading let binds the parameters.

Parameters can be types, values, predicates, or functors.

Predicate and functor parameters cannot occur in attr, mode, struct, func, or pred items.

Some templates allow short forms such as `Module over R` and `Subset of X`.
Constrained templates require brackets.

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.3 - The Evo Answer: Templates

7.3 - The Evo Answer: Templates

This template begins with a type parameter specification example

```

definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;

```

A template is an ordinary definition block. Its leading `let` binds the parameters.
 Parameters can be types, values, predicates, or functors.
 Predicate and functor parameters cannot occur in `attr`, `mode`, `struct`, `func`, or `pred` items.
 Some templates allow short forms such as `Module over R` and `Subset of X`.
 Constrained templates require brackets.

Read aloud:

- **This template begins with a type parameter.**
- **A template is an ordinary definition block. Its leading `let` binds the parameters.**
- **Parameters can be types, values, predicates, or functors.**
- **Predicate and functor parameters cannot occur in `attr`, `mode`, `struct`, `func`, or `pred` items.**
- Some templates allow short forms such as `Module over R` and `Subset of X`.
Constrained templates require brackets.

Optional notes and sources:

- The short forms display `Module [R]` and `Subset [X]`.

specification example

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

The bound requires commutativity, associativity, and a unit.
Product[T] starts with the unit and folds the sequence with the selected operation.
The library proves these defining equations.

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.4 - Bounded Parameters And Generic Theorems

deep dive

Mizar Evo [specification example](#) (§18.2.2; proof omitted)

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Message:

The bound requires commutativity, associativity, and a unit.
Product[T] starts with the unit and folds the sequence with the selected operation.
The library proves these defining equations.

Read aloud:

- The bound requires commutativity, associativity, and a unit.
- Product [T] starts with the unit and folds the sequence with the selected operation. The library proves these defining equations.

specification example

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]

let R be commutative Ring;
PermProduct[R qua AddMagma]           :: R's additive view
PermProduct[R qua MulMagma]          :: R's multiplicative view
```

A ring reaches Magma along two paths. `qua` selects the view. The view also sets the notation: the generic `*` appears as `+` for addition.

The required attributes must hold on the selected view.

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.5 - One Proof, Many Instantiations

deep dive

7.5 - One Proof, Many Instantiations

deep dive

Instantiation `specification example` (with the required registrations)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]

let R be commutative Ring;
PermProduct[R qua AddMagma]      :: R's additive view
PermProduct[R qua MulMagma]     :: R's multiplicative view
```

Message:

A ring reaches Magma along two paths. qua selects the view. The view also sets the notation: the generic * appears as + for addition.
The required attributes must hold on the selected view.

Read aloud:

- A ring reaches Magma along two paths. qua selects the view. The view also sets the notation: the generic * appears as + for addition.
- The required attributes must hold on the selected view.

7.6 - Schemes Become Ordinary Templates

deep dive

specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

predicate parameters follow the familiar defpred convention;
legacy schemes have a direct, mechanical migration target;
theorem instantiation uses explicit brackets, so tools and artifacts see exactly which instance a proof uses.

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.6 - Schemes Become Ordinary Templates

deep dive

Mizar Evo **specification example**

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

Key Points:

predicate parameters follow the familiar defpred convention;
legacy schemes have a direct, mechanical migration target;
theorem instantiation uses explicit brackets, so tools and artifacts see exactly which instance a proof uses.

Read aloud:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- theorem instantiation uses explicit brackets, so tools and artifacts see exactly which instance a proof uses.

7.7 - What Is Preserved, What We Ask

Scheme-style reasoning keeps the same power.

The words `of` and `over` help keep mathematical text readable.

Each instantiation is checked. Templates add no new logic.

I would like to try this design on existing MML schemes. The next story connects proofs with algorithms.

Which MML schemes should be the first migration targets?

Are brackets acceptable as the canonical identity form, with `of/over` as display forms?

For `func` and `pred` templates, normalized declared argument types must determine each inferred type parameter uniquely. `qua` views are never inferred. Does this rule require too many explicit `[T]` arguments?

Mizar Evo

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.7 - What Is Preserved, What We Ask

7.7 - What Is Preserved, What We Ask

Scheme-style reasoning keeps the same power.
The words `of` and `over` help keep mathematical text readable.
Each instantiation is checked. Templates add no new logic.
I would like to try this design on existing MML schemes. The next story connects proofs with algorithms.

Questions for later review:

Which MML schemes should be the first migration targets?
Are brackets acceptable as the canonical identity form, with `of/over` as display forms?
For `func` and `pred` templates, normalized declared argument types must determine each inferred type parameter uniquely. `qua` views are never inferred. Does this rule require too many explicit `[T]` arguments?

Read aloud:

- Scheme-style reasoning keeps the same power.
- The words `of` and `over` help keep mathematical text readable.
- **Each instantiation is checked. Templates add no new logic.**
- **I would like to try this design on existing MML schemes. The next story connects proofs with algorithms.**

8.1 - The Problem

Mizar already provides arithmetic automation and proofs of program correctness.

Evo proposes a language form for algorithms with contracts.

The goal is to connect verification, execution, and code extraction within one language and toolchain.

Execution and code extraction are still planned work.

Connect mathematical proofs with executable algorithms.

Mizar Evo

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.1 - The Problem

8.1 - The Problem

Mizar already provides arithmetic automation and proofs of program correctness.

Evo proposes a language form for algorithms with contracts.

The goal is to connect verification, execution, and code extraction within one language and toolchain.

Execution and code extraction are still planned work.

Key Phrase:

Connect mathematical proofs with executable algorithms.

Examples in MML: NUMERALS for arithmetic requirements; FIB_FUSC for program correctness (see the Formalized Mathematics contents (<https://mizar.uwb.edu.pl/fm/contents.html>)).

Read aloud:

- **Mizar already provides arithmetic automation and proofs of program correctness.**
- **Evo proposes a language form for algorithms with contracts.**
- The goal is to connect verification, execution, and code extraction within one language and toolchain.
- **Execution and code extraction are still planned work.**
- Connect mathematical proofs with executable algorithms.

Optional notes and sources:

- Examples in MML: NUMERALS for arithmetic requirements; FIB_FUSC for program correctness (see the Formalized Mathematics contents (<https://mizar.uwb.edu.pl/fm/contents.html>)).

8.2 - The Evo Answer: Algorithms With Contracts

This is Euclid's algorithm with a contract **specification example**

```
definition
  let a, b be Nat;
  terminating algorithm euclid_gcd(a, b) -> Nat
    requires a >= 1 & b >= 1
    ensures result = Gcd(a, b)
  do
    var x := a; var y := b;
    while y <> 0 do
      invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);
      decreasing y;
      const r := x mod y; x := y; y := r;
    end;
    return x;
  end
end
```

The
loop computes the result. The decreasing measure on y proves termination.

Mizar Evo

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.2 - The Evo Answer: Algorithms With Contracts

8.2 - The Evo Answer: Algorithms With Contracts

This is Euclid's algorithm with a contract specification example

```

definition
  let a, b be Nat;
  terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);
    decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;

```

The contract and invariant use mathematical Gcd, so there is no circular definition. **The loop computes the result. The decreasing measure on y proves termination.**

Read aloud:

- **This is Euclid's algorithm with a contract.**
- The contract and invariant use mathematical Gcd, so there is no circular definition. **The loop computes the result. The decreasing measure on y proves termination.**

Optional notes and sources:

- This example is condensed from specification section 20.12.

8.3 - The Evo Answer: Proof By Computation

The specification allows proofs by computation specification example

```
theorem EuclidGcd12_8: euclid_gcd(12, 8) = 4 by computation;  
theorem EuclidGcd100_75: euclid_gcd(100, 75) = 25 by computation;  
  
theorem Fact10: factorial(10) = 3628800  
proof  
  thus thesis by computation(steps: 100000);  
end;
```

The Mizar Virtual Machine, or MVM, evaluates ground equalities and predicates. It uses computable algorithms defined in the current package. Non-ground formulas need classical proofs.

Step, time, and depth limits are optional. Each defaults to zero, meaning unlimited. **Other packages' algorithm bodies cannot run here. Their ensures contracts can be used when termination is known.**

Mizar Evo

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.3 - The Evo Answer: Proof By Computation

8.3 - The Evo Answer: Proof By Computation

The specification allows proofs by computation specification example

```
theorem EuclidGcd12_8: euclid_gcd(12, 8) = 4 by computation;  
theorem EuclidGcd100_75: euclid_gcd(100, 75) = 25 by computation;  
  
theorem Fact10: factorial(10) = 3628800  
proof  
  thus thesis by computation(steps: 100000);  
end;
```

The Mizar Virtual Machine, or MVM, evaluates ground equalities and predicates. It uses computable algorithms defined in the current package. Non-ground formulas need classical proofs. Step, time, and depth limits are optional. Each defaults to zero, meaning unlimited. Other packages' algorithm bodies cannot run here. Their ensures contracts can be used when termination is known.

Read aloud:

- **The specification allows proofs by computation.**
- **The Mizar Virtual Machine, or MVM, evaluates ground equalities and predicates. It uses computable algorithms defined in the current package. Non-ground formulas need classical proofs.**
- Step, time, and depth limits are optional. Each defaults to zero, meaning unlimited.
- **Other packages' algorithm bodies cannot run here. Their ensures contracts can be used when termination is known.**

specification example

```
definition
  let n be Nat;
  terminating algorithm factorial(n) -> Nat
    decreasing n
  do
    if n = 0 do return 1; end;
    return n * factorial(n - 1);
  end
end
```

ordinary func definitions are definitional extensions: never recursive;
a terminating algorithm becomes a functor after all contract and termination obligations are proved for inputs satisfying requires;
this is the only way to add recursion to the mathematical layer. It requires a proof.

Mizar Evo

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.4 - The Evo Answer: Termination Allows Recursion

deep dive

Mizar Evo specification example

```
definition
  let n be Nat;
  terminating algorithm factorial(n) -> Nat
  decreasing n
  do
    if n = 0 do return 1; end;
    return n * factorial(n - 1);
  end;
end;
```

Key Points:

ordinary func definitions are definitional extensions: never recursive;
a terminating algorithm becomes a functor after all contract and termination obligations are proved for inputs satisfying requires;
this is the only way to add recursion to the mathematical layer. It requires a proof.

Read aloud:

- ordinary func definitions are definitional extensions: never recursive;
- a terminating algorithm becomes a functor after all contract and termination obligations are proved for inputs satisfying requires;
- this is the only way to add recursion to the mathematical layer. It requires a proof.

Optional notes and sources:

- Each call must satisfy requires. An ordinary algorithm may prove only partial correctness and is not promoted to a functor.

contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
by `computation` uses MVM replay with optional limits;
code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.

Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

Mizar Evo

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.5 - Computation Never Redefines Truth

deep dive

Key Points:

contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
by `computation` uses MVM replay with optional limits;
code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.

Message:

Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

Read aloud:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by `computation` uses MVM replay with optional limits;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.
- Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

8.6 - What Is Preserved, What We Ask

Algorithms stay inside definition blocks.

Verified promotion lets a terminating algorithm become a functor after its obligations are proved.

Proofs can also use `by computation` for computable ground calls in the current package.

A partial algorithm's ensures requires evidence that the call terminates.

The first-order set-theoretic foundations stay the same.

Now I will turn to how we publish and cite this work.

Which computational examples would show clear benefits to mathematicians without shifting the culture toward programming?

Are there MML areas (number theory, combinatorics, finite structures) where `by computation` would immediately shorten real proofs?

Which extraction targets matter first, if any?

Mizar Evo

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.6 - What Is Preserved, What We Ask

8.6 - What Is Preserved, What We Ask

Algorithms stay inside definition blocks.

Verified promotion lets a terminating algorithm become a functor after its obligations are proved.

Proofs can also use `by computation` for computable ground calls in the current package.

A partial algorithm's `ensures` requires evidence that the call terminates.

The first-order set-theoretic foundations stay the same.

Now I will turn to how we publish and cite this work.

Questions for later review:

Which computational examples would show clear benefits to mathematicians without shifting the culture toward programming?

Are there MML areas (number theory, combinatorics, finite structures) where by computation would immediately shorten real proofs?

Which extraction targets matter first, if any?

Read aloud:

- Algorithms stay inside definition blocks.
- **Verified promotion lets a terminating algorithm become a functor after its obligations are proved.**
- Proofs can also use `by computation` for computable ground calls in the current package.
- **A partial algorithm's `ensures` requires evidence that the call terminates.**
- The first-order set-theoretic foundations stay the same.
- **Now I will turn to how we publish and cite this work.**

9.1 - The Problem

Formalized Mathematics links a research journal to a formal library. People can read and cite its articles.

A journal article has an order that helps explain the mathematics. A reusable library needs an order based on dependencies.

Changes to library organization can therefore affect links to published articles. Package reuse also lacks an identity for journal citations.

Evo proposes separate, linked records for journal articles and library items.

Mizar Evo

└ Part 9. Story 8: A Library You Can Cite

└ 9.1 - The Problem

9.1 - The Problem

Formalized Mathematics links a research journal to a formal library. People can read and cite its articles.

A journal article has an order that helps explain the mathematics. A reusable library needs an order based on dependencies.

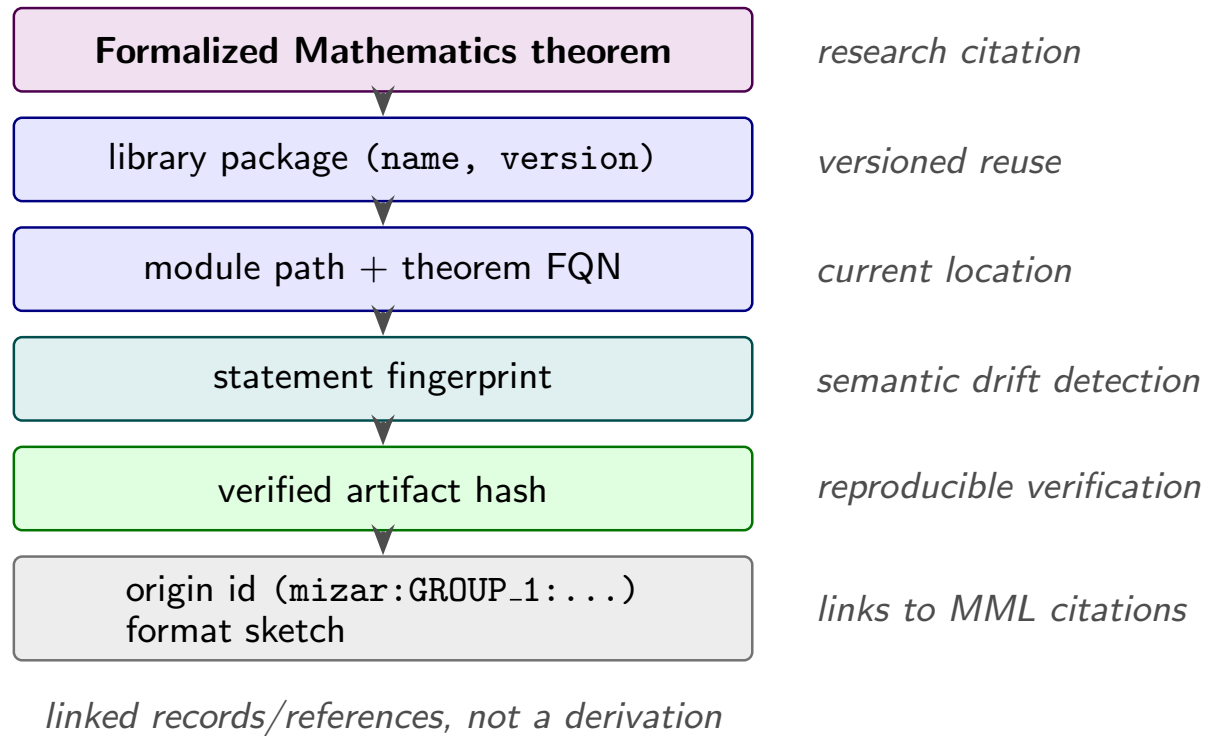
Changes to library organization can therefore affect links to published articles. Package reuse also lacks an identity for journal citations.

Evo proposes separate, linked records for journal articles and library items.

Read aloud:

- Formalized Mathematics links a research journal to a formal library. People can read and cite its articles.
- **A journal article has an order that helps explain the mathematics. A reusable library needs an order based on dependencies.**
- **Changes to library organization can therefore affect links to published articles. Package reuse also lacks an identity for journal citations.**
- **Evo proposes separate, linked records for journal articles and library items.**

9.2 - The Evo Proposal: Linked Records



This sketch links citations, library items, verification artifacts, and MML origins.

It shows links between records, not a derivation.

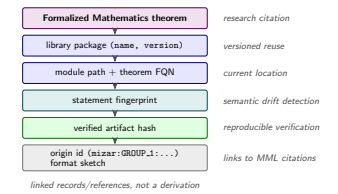
2026-09-17

Mizar Evo

└ Part 9. Story 8: A Library You Can Cite

└ 9.2 - The Evo Proposal: Linked Records

9.2 - The Evo Proposal: Linked Records



This sketch links citations, library items, verification artifacts, and MML origins.
It shows links between records, not a derivation.

Read aloud:

- This sketch links citations, library items, verification artifacts, and MML origins.
- It shows links between records, not a derivation.

9.3 - Who Gains What

deep dive

2026-09-17

Mizar Evo

└─ Part 9. Story 8: A Library You Can Cite

└─ 9.3 - Who Gains What

deep dive

Optional detail (see slide).

Audience	Gain
readers	written explanations come first; formal source is one click away
maintainers	refactoring no longer rewrites published explanations
authors	frozen pub article identities; FQNs for library locations
planned AI tools	prose for retrieval, fingerprints for exact context

9.4 - What Is Preserved, What We Ask

Formalized Mathematics would remain a journal with review and written explanations.

Origin metadata would link migrated items to their MML citations. The migration mapping still needs to be defined.

I would like to discuss which identifiers readers should see in citations.

That completes the eight stories. Let us now look at the whole design.

Which identity should be primary in user-facing citations: article label, library FQN, or origin id?

How should existing Formalized Mathematics articles link to migrated modules - by updating old links now, when articles are revised, or not at all?

Mizar Evo

└ Part 9. Story 8: A Library You Can Cite

└ 9.4 - What Is Preserved, What We Ask

9.4 - What Is Preserved, What We Ask

Formalized Mathematics would remain a journal with review and written explanations.

Origin metadata would link migrated items to their MML citations. The migration mapping still needs to be defined.

I would like to discuss which identifiers readers should see in citations.

That completes the eight stories. Let us now look at the whole design.

Questions for later review:

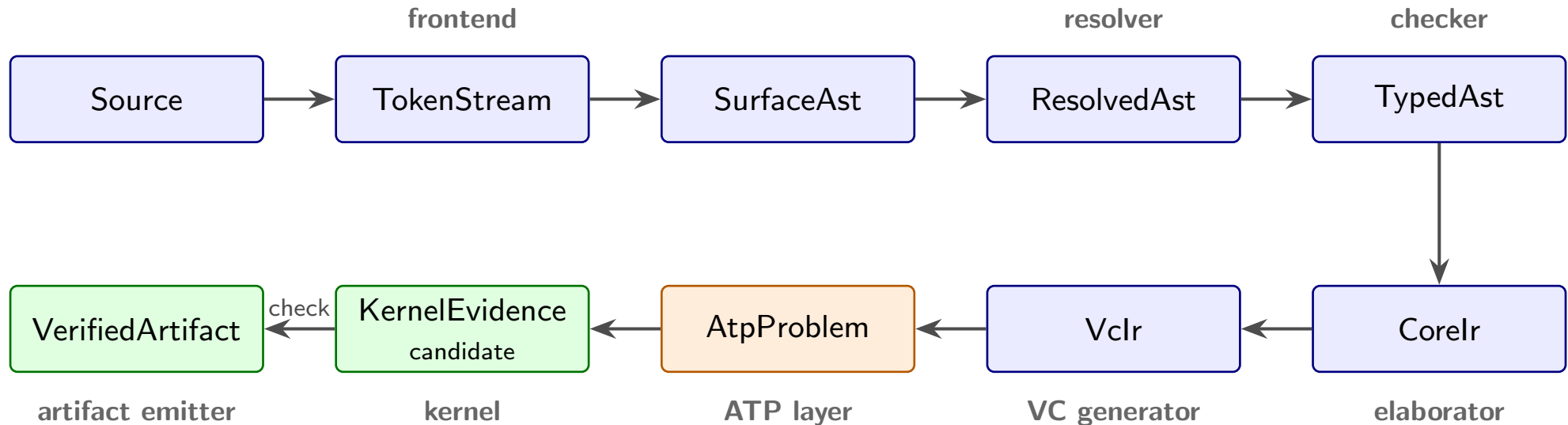
Which identity should be primary in user-facing citations: article label, library FQN, or origin id?

How should existing Formalized Mathematics articles link to migrated modules - by updating old links now, when articles are revised, or not at all?

Read aloud:

- Formalized Mathematics would remain a journal with review and written explanations.
- **Origin metadata would link migrated items to their MML citations. The migration mapping still needs to be defined.**
- I would like to discuss which identifiers readers should see in citations.
- **That completes the eight stories. Let us now look at the whole design.**

10.1 - The Core ATP Path



This diagram shows the ATP path through the proposed pipeline. Only obligations still open after deterministic discharge go to ATP. Earlier discharge also needs evidence.

Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

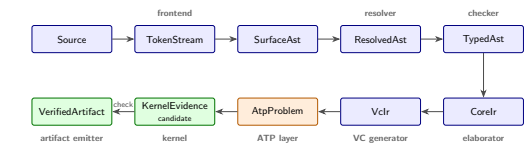
2026-09-17

Mizar Evo

└ Part 10. Architecture In One Picture

└ 10.1 - The Core ATP Path

10.1 - The Core ATP Path



This diagram shows the ATP path through the proposed pipeline. Only obligations still open after deterministic discharge go to ATP. Earlier discharge also needs evidence. Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

Read aloud:

- This diagram shows the ATP path through the proposed pipeline.
- Only obligations still open after deterministic discharge go to ATP. Earlier discharge also needs evidence.
- Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

10.2 - Responsibility Split

deep dive

2026-09-17

Mizar Evo

└─ Part 10. Architecture In One Picture

└─ 10.2 - Responsibility Split

deep dive

Optional detail (see slide).

Layer	Responsibility
frontend	lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP layer plus kernel	untrusted search, then acceptance by checking
artifact emitter	stable outputs for tools and dependents

10.3 - Pipeline Stages For The Eight Stories

These eight stories belong to one pipeline, from name resolution to publication. Each stage keeps its own responsibility for meaning and proof checking.

Mizar Evo

└ Part 10. Architecture In One Picture

└ 10.3 - Pipeline Stages For The Eight Stories

10.3 - Pipeline Stages For The Eight Stories

These eight stories belong to one pipeline, from name resolution to publication.
Each stage keeps its own responsibility for meaning and proof checking.

Details for later review:

Story	Pipeline stage
dependencies	resolver, package manager
structures and automation	checker (inheritance and cluster graphs, traces)
search vs trust	ATP layer, kernel, formula/substitution evidence
scale	artifacts, fingerprints, scheduler
templates	elaborator (checked instantiation)
algorithms	VC generator, MVM
publication	artifact emitter, doc generation

Read aloud:

- These eight stories belong to one pipeline, from name resolution to publication.
- Each stage keeps its own responsibility for meaning and proof checking.

*Reject what must not pass
before
Accept everything that should pass*

soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
with these tests in place, we add coverage for accepted language forms.

2026-09-17

Mizar Evo

└ Part 10. Architecture In One Picture

└ 10.4 - Testing The Trust Boundary

deep dive

10.4 - Testing The Trust Boundary

deep dive

Key Phrase:

*Reject what must not pass
before
Accept everything that should pass*

Key Points:

soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
with these tests in place, we add coverage for accepted language forms.

Read aloud:

- Reject what must not pass before Accept everything that should pass
- soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
- with these tests in place, we add coverage for accepted language forms.

11.0 - Where The Project Stands Today

Before I finish, let me separate the design from the implementation.

The specification has 24 chapters and appendices. English is canonical; Japanese companions are available.

Implemented parts include frontend processing, selected semantic cases, ATP candidate generation, and kernel evidence checking.

The full path from source through ATP and kernel checking to published artifacts is still in progress. Tests with real external provers are still needed. MVM execution, code extraction, and full MML migration remain future work.

Mizar Evo

└ Part 11. Roadmap And Collaboration

└ 11.0 - Where The Project Stands Today

11.0 - Where The Project Stands Today

Before I finish, let me separate the design from the implementation.
The specification has 24 chapters and appendices. English is canonical; Japanese companions are available.
Implemented parts include frontend processing, selected semantic cases, ATP candidate generation, and kernel evidence checking.
The full path from source through ATP and kernel checking to published artifacts is still in progress. Tests with real external provers are still needed.
MVM execution, code extraction, and full MML migration remain future work.

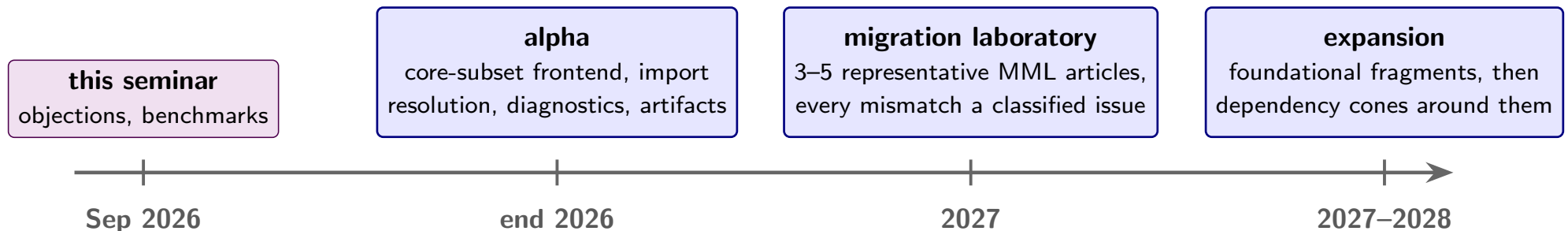
Read aloud:

- **Before I finish, let me separate the design from the implementation.**
- The specification has 24 chapters and appendices. English is canonical; Japanese companions are available.
- **Implemented parts include frontend processing, selected semantic cases, ATP candidate generation, and kernel evidence checking.**
- **The full path from source through ATP and kernel checking to published artifacts is still in progress. Tests with real external provers are still needed.**
- **MVM execution, code extraction, and full MML migration remain future work.**

Optional notes and sources:

- This is the September 2026 scope. Component tests do not show that the full pipeline works. See `doc/design/todo.md`, Completion Gates.

11.1 - Migration Needs Research



We plan an alpha at the end of 2026. It covers a core frontend subset, import and module resolution prototypes, diagnostics, and early artifacts. In 2027, we plan to translate three to five representative MML articles by hand and by script. We will record every mismatch.

In 2027 and 2028, we plan to expand from set and relation fragments to algebraic structures and their dependencies.

The alpha does not aim at full MML verification, a final compatibility layer, or a stable AI protocol.

Mizar Evo

└ Part 11. Roadmap And Collaboration

└ 11.1 - Migration Needs Research

11.1 - Migration Needs Research



We plan an alpha at the end of 2026. It covers a core frontend subset, import and module resolution prototypes, diagnostics, and early artifacts. In 2027, we plan to translate three to five representative MML articles by hand and by script. We will record every mismatch. In 2027 and 2028, we plan to expand from set and relation fragments to algebraic structures and their dependencies. The alpha does not aim at full MML verification, a final compatibility layer, or a stable AI protocol.

Read aloud:

- We plan an alpha at the end of 2026. It covers a core frontend subset, import and module resolution prototypes, diagnostics, and early artifacts.
- In 2027, we plan to translate three to five representative MML articles by hand and by script. We will record every mismatch.
- In 2027 and 2028, we plan to expand from set and relation fragments to algebraic structures and their dependencies.
- The alpha does not aim at full MML verification, a final compatibility layer, or a stable AI protocol.

translated articles and lines; accepted parser subset;
resolved imports versus unresolved dependencies;
obligations closed deterministically versus by ATP with kernel evidence;
memory and wall-clock per module, incremental versus clean;
compatibility decisions that required human judgment.

2026-09-17

Mizar Evo

└ Part 11. Roadmap And Collaboration

└ 11.2 - What We Will Measure

deep dive

11.2 - What We Will Measure

deep dive

Key Points:

translated articles and lines; accepted parser subset;
resolved imports versus unresolved dependencies;
obligations closed deterministically versus by ATP with kernel evidence;
memory and wall-clock per module, incremental versus clean;
compatibility decisions that required human judgment.

Read aloud:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP with kernel evidence;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

- define origin mappings to preserve MML theorem identity;
- keep compatibility aliases where they help migration;
- record every difference from old behavior with a reason and a test.

Mizar Evo

└ Part 11. Roadmap And Collaboration

└ 11.3 - Compatibility Policy And Risks

deep dive

Policy:

define origin mappings to preserve MML theorem identity;
keep compatibility aliases where they help migration;
record every difference from old behavior with a reason and a test.

Risk	How to reduce it
compatibility work takes all our time	small representative parts first; no all-at-once translation
registrations behave differently	trace artifacts and comparison reports early
package layout breaks journal links	origin metadata, article-to-library identifiers
AI edits hide migration mistakes	Red edits stay forbidden to ordinary agents; verifier artifacts required

Read aloud:

- define origin mappings to preserve MML theorem identity;
- keep compatibility aliases where they help migration;
- record every difference from old behavior with a reason and a test.

11.4 - What September 2026 Should Produce

For this visit, I hope we can choose migration benchmark articles in priority order.
I would like to agree on the compatibility metadata we need.
I also hope to collect comments on the eight stories, especially structures and clusters.
A first paper outline and a shared list of objections would help guide the next steps.

2026-09-17

Mizar Evo

└ Part 11. Roadmap And Collaboration

└ 11.4 - What September 2026 Should Produce

11.4 - What September 2026 Should Produce

For this visit, I hope we can choose migration benchmark articles in priority order.
I would like to agree on the compatibility metadata we need.
I also hope to collect comments on the eight stories, especially structures and clusters.
A first paper outline and a shared list of objections would help guide the next steps.

Read aloud:

- For this visit, I hope we can choose migration benchmark articles in priority order.
- I would like to agree on the compatibility metadata we need.
- I also hope to collect comments on the eight stories, especially structures and clusters.
- A first paper outline and a shared list of objections would help guide the next steps.

11.5 - What We Ask Of You

I do not expect us to settle every design question today.

The handout keeps the questions below for later review.

Your choice of small MML examples would help us test the proposal in practice.

Which MML articles are small but structurally representative?

Which idioms matter to the Mizar community, beyond their technical use?

Which of the eight stories is most wrong, and why?

What migration result would convince the community that Evo is a serious project?

2026-09-17

Mizar Evo

└ Part 11. Roadmap And Collaboration

└ 11.5 - What We Ask Of You

11.5 - What We Ask Of You

I do not expect us to settle every design question today.
The handout keeps the questions below for later review.
Your choice of small MML examples would help us test the proposal in practice.

Questions for later review:

Which MML articles are small but structurally representative?
Which idioms matter to the Mizar community, beyond their technical use?
Which of the eight stories is most wrong, and why?
What migration result would convince the community that Evo is a serious project?

Read aloud:

- I do not expect us to settle every design question today.
- The handout keeps the questions below for later review.
- Your choice of small MML examples would help us test the proposal in practice.

12.1 - The Main Question, Again

I would like to finish with the question from the beginning.

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Which small MML articles should we migrate first?
Which proposal needs the most change?

Mizar Evo

└ Part 12. Closing

└ 12.1 - The Main Question, Again

12.1 - The Main Question, Again

I would like to finish with the question from the beginning.

Slide question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Which small MML articles should we migrate first?
Which proposal needs the most change?

Read aloud:

- I would like to finish with the question from the beginning.
- What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?
- Which small MML articles should we migrate first?
- Which proposal needs the most change?

12.2 - Closing

**Mizar Evo should update the parts that need to support a larger library.
It should keep the parts that define Mizar's mathematical identity.
Thank you for listening. I would welcome your questions and comments.**

2026-09-17

Mizar Evo

└ Part 12. Closing

└ 12.2 - Closing

12.2 - Closing

Mizar Evo should update the parts that need to support a larger library.
It should keep the parts that define Mizar's mathematical identity.
Thank you for listening. I would welcome your questions and comments.

Read aloud:

- **Mizar Evo should update the parts that need to support a larger library.**
- **It should keep the parts that define Mizar's mathematical identity.**
- **Thank you for listening. I would welcome your questions and comments.**

Backup A. Prepared Exact Examples (1/2)

ALGSTR_0: https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz
STRUCT_0: https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz
NAT_1: https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz

Mizar Evo

└ Backup A. Prepared Exact Examples

└ Backup A. Prepared Exact Examples (1/2)

Backup A. Prepared Exact Examples (1/2)

Exact excerpts used in this talk:

Purpose	Source	Lines	Used in
Structure definition	algstr_0.miz	37-40	Frames 0.2, 3.1
Article environment	algstr_0.miz	15-25	Frame 2.1
Registration/cluster	algstr_0.miz	104-109	Frame 4.1

Exact excerpts used in this talk:

Purpose	Source	Lines	Used in
Proof citation	struct_0.miz	637-643	Frame 5.4
Induction scheme	nat_1.miz	near 90	Frame 7.1

Source URLs:

ALGSTR_0: https://mizar.uwb.edu.pl/version/current/mml/algstr_0.mizSTRUCT_0: https://mizar.uwb.edu.pl/version/current/mml/struct_0.mizNAT_1: https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz

Read aloud:

- ALGSTR_0: https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz
- STRUCT_0: https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz
- NAT_1: https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz

Backup A. Prepared Exact Examples (2/2)

MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

2026-09-17

Mizar Evo

└ Backup A. Prepared Exact Examples

└ Backup A. Prepared Exact Examples (2/2)

Backup A. Prepared Exact Examples (2/2)

Attribution note:

MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Read aloud:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Backup B. Specification Reference Map (1/2)

Mizar Evo

└ Backup B. Specification Reference Map

└ Backup B. Specification Reference Map (1/2)

Backup B. Specification Reference Map (1/2)

Specification used: 10 September 2026. For grammar and semantics, see [doc/spec/en/00.index.md](https://github.com/aabaa/mizar-evo) in the repository (<https://github.com/aabaa/mizar-evo>). Online files may change after this talk.

Topic	Specification source (under doc/spec/en/)
Modules and imports	12.modules_and_namespaces.md
Structures and inheritance	06.structures.md
Attributes and modes	06.attributes.md, 07.modes.md
Registrations and reductions	17.clusters_and_registrations.md
Templates and schemes	18.templates.md
Algorithms and MVM	20.algorithm_and_verification.md
ATP and kernel evidence	21.source_code_annotation_and_atp.md

Read aloud:

- Specification used: 10 September 2026. For grammar and semantics, see [doc/spec/en/00.index.md](https://github.com/aabaa/mizar-evo) in the repository (<https://github.com/aabaa/mizar-evo>). Online files may change after this talk.

Backup B. Specification Reference Map (2/2)

2026-09-17

Mizar Evo

└ Backup B. Specification Reference Map

└ Backup B. Specification Reference Map (2/2)

Optional detail (see slide).

Backup B. Specification Reference Map (2/2)

Topic	Specification source (under <i>doc/spec/en/</i>)
Packages and artifacts	23.package_management_and_build_system.md
Cross-chapter grammar	appendix_a.grammar_summary.md
Worked library sketches	sample_codes.md

Backup D. Possible Paper Outline (1/2)

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Introduction: why Mizar needs evolution now.

Mizar as baseline: readability, MML, Formalized Mathematics.

Design principles and the three goals.

Language evolution: dependencies, structures, registrations, templates.

Verifier architecture, kernel evidence, and the trusted SAT checker.

Verified computation and the MVM.

AI-safe proof development.

2026-09-17

Mizar Evo

└ Backup D. Possible Paper Outline

└ Backup D. Possible Paper Outline (1/2)

Backup D. Possible Paper Outline (1/2)

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

Introduction: why Mizar needs evolution now.
Mizar as baseline: readability, MML, Formalized Mathematics.
Design principles and the three goals.
Language evolution: dependencies, structures, registrations, templates.
Verifier architecture, kernel evidence, and the trusted SAT checker.
Verified computation and the MVM.
AI-safe proof development.

Read aloud:

- Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics
- Introduction: why Mizar needs evolution now.
- Mizar as baseline: readability, MML, Formalized Mathematics.
- Design principles and the three goals.
- Language evolution: dependencies, structures, registrations, templates.
- Verifier architecture, kernel evidence, and the trusted SAT checker.
- Verified computation and the MVM.
- AI-safe proof development.

Backup D. Possible Paper Outline (2/2)

Package-based library and publication workflow.

Migration plan and evaluation measures.

Related work and plans for working together.

2026-09-17

Mizar Evo

└ Backup D. Possible Paper Outline

└ Backup D. Possible Paper Outline (2/2)

Backup D. Possible Paper Outline (2/2)

Possible sections:

Package-based library and publication workflow.
Migration plan and evaluation measures.
Related work and plans for working together.

Read aloud:

- Package-based library and publication workflow.
- Migration plan and evaluation measures.
- Related work and plans for working together.

Backup F. Possible Objections (1/2)

2026-09-17

Mizar Evo

└ Backup F. Possible Objections

└ Backup F. Possible Objections (1/2)

Optional detail (see slide).

Optional notes and sources:

- Use these answers only if someone asks these questions.

Backup F. Possible Objections (1/2)

Prepared answers to other possible objections:

Objection	Prepared answer
Why not improve current Mizar incrementally?	Some proposals may also help existing tools. Evo explores changes to modules, proof evidence, and library artifacts together. Migration examples should help us decide which changes are useful.
What happens to authorship and credit of MML articles?	We propose origin mappings that preserve identity and credit during migration. The mapping format is still open. The pub namespace keeps published articles unchanged.
Is this a fork of the community?	It is a proposal to the community; this visit is its first review. The namespace governance model assumes that the Mizar team controls the <code>mm1</code> root.

Backup F. Possible Objections (2/2)

2026-09-17

Mizar Evo

└ Backup F. Possible Objections

└ Backup F. Possible Objections (2/2)

Optional detail (see slide).

Backup F. Possible Objections (2/2)

Prepared answers to other possible objections:

Objection	Prepared answer
What about GPL / CC-BY-SA obligations?	Migration preserves license and attribution of MML content; toolchain licensing is open for discussion.
Are the claims about AI too strong?	AI assistance is an optional layer that never enters the trusted base; every proposal also works without it.
Why a new kernel instead of the existing checker?	Formula/substitution evidence needs a small trusted core, including a SAT checker. Mizar Evo's specification defines the obligations; legacy behavior guides migration comparisons.